

Ejercitación Grafos

Algoritmos y Estructuras de Datos II

Alumno: Matías Cúneo

Parte 2

12.

Como G es un árbol, es conexo y acíclico. En particular, entre u y v existe un único camino simple P .

Al agregar la arista ($e = \{u, v\}$), dicho camino P junto con e forma un ciclo. Por lo tanto, el nuevo grafo contiene al menos un ciclo.

Además, no puede formarse ningún otro ciclo. En efecto, cualquier ciclo que contenga la nueva arista e debe utilizar, además, un camino entre u y v compuesto únicamente por aristas de G . Pero en un árbol existe un único camino entre u y v . Por consiguiente, el ciclo formado es único.



13.

Sea $G = (V, E)$ un grafo no dirigido y sea T un árbol BFS de G , construido a partir de una raíz s . Si una arista $(u, v) \in E$ no pertenece a T , entonces los niveles de u y v en T difieren a lo sumo 1, es decir: $|Nivel(u) - Nivel(v)| \leq 1$ (1)

Sea $\delta(s, x)$ la distancia desde la raíz s hasta un vértice x . Por propiedad de árboles: $Nivel(x) = \delta(s, x)$.

Como $(u, v) \in E$, existe un camino desde s hasta v obtenido tomando un camino mínimo desde s hasta u y agregando la arista (u, v) . Por lo tanto $\delta(s, v) \leq \delta(s, u) + 1$.

Análogamente, $\delta(s, u) \leq \delta(s, v) + 1$. De ambas desigualdades sigue que $|\delta(s, u) - \delta(s, v)| \leq 1$.

Como las distancias desde s coinciden con los niveles del árbol BFS, concluimos (1)



Parte 3

16.

Sea T un árbol abarcador de costo mínimo de G . Si $(u, v) \in T$, no hay nada que demostrar.

Si $(u, v) \notin T$, al agregar la arista a T se forma un ciclo. Como (u, v) cruza el corte $\{U, V - U\}$, este ciclo debe contener alguna otra arista (x, y) que también cruce dicho corte.

Por hipótesis, $w(u, v) \leq w(x, y)$.

Ahora, si eliminamos (x, y) del ciclo, el grafo resultante $T' = T - \{(x, y)\} + \{(u, v)\}$ sigue siendo un árbol abarcador, además, $w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$.

Como T era un árbol abarcador de costo mínimo, $w(T')$ no puede ser menor que $w(T)$. Por lo tanto, $w(T') = w(T)$ y T' también es un AACM.

Como $(u, v) \in T'$, concluimos que (u, v) pertenece a algún AACM de G .

■

17.

Sea T un árbol abarcador de costo mínimo de G , si $e \notin T$, entonces $T \subseteq (V, A - e)$ y no hay nada que demostrar.

Supongamos que $e \in T$, al eliminar e de T , el árbol se divide en dos componentes, cuyos conjuntos de vértices van a ser $\{U\}$ y $\{V - U\}$.

Como e pertenece a un ciclo C y conecta un vértice de U con uno de $V - U$, el ciclo debe contener alguna otra arista $f \neq e$ que también cruce el corte $\{U, V - U\}$. Al ser e una arista de mayor costo del ciclo, $w(f) \leq w(e)$.

Entonces podemos definir $T' = T - \{e\} + \{f\}$. Como f conecta las dos componentes creadas al eliminar e , T' vuelve a ser un árbol abarcador. Además, $w(T') = w(T) - w(e) + w(f) \leq w(T)$.

Pero T era un AACM, por lo que no puede existir otro árbol abarcador de costo mínimo de menor costo. Por lo tanto, $w(T') = w(T)$ y T' también es un AACM tal que $e \notin T'$, por lo que $T' \subseteq (V, A - e)$.

■

18.

Teniendo T_1 y T_2 árboles abarcadores de costo mínimo de dos componentes conexas disjuntas $G_1 = (V_1, A_1)$ y $G_2 = (V_2, A_2)$ y $e = (u, v)$ una arista de costo mínimo entre V_1 y V_2 con $u \in V_1$ y $v \in V_2$.

Como T_1 y T_2 son árboles, cada uno es conexo y acíclico. Al agregar la arista e , el grafo $T = T_1 \cup T_2 \cup \{e\}$ resulta conexo y no se forma ningún ciclo. Por lo tanto, T es un árbol abarcador.

Sea S cualquier otro árbol abarcador del grafo resultante, su restricción a cada componente debe tener al menos costo $w(T_1)$ y $w(T_2)$ respectivamente. Además, debe contener al menos una arista que conecte V_1 con V_2 . Como e es una arista de costo mínimo entre ambas componentes, el costo de dicha arista es al menos $w(e)$.

Por lo tanto, $w(S) \geq w(T_1) + w(T_2) + w(e) = w(T)$, entonces ningún árbol abarcador puede tener menor costo que T , luego T es un AACM.



19.

- a. Para encontrar el árbol abarcador de costo máximo, habría que invertir la lógica de búsqueda de costo entre los conjuntos $\{U\}$ y $\{V - U\}$:

```
if v not in U and w > max_weight:
    max_weight = w
    max_v = v
    max_u = u
```

- b. Para obtener un árbol abarcador cualquiera, habría que seleccionar aristas de manera aleatoria entre los dos conjuntos, no necesariamente la de costo mínimo.

- c. Habría que sumar la condición de revisar si la arista existe en E y que dicho criterio tome prioridad por sobre encontrar la arista de costo mínimo en la iteración actual:

```
if (u, v, w) in E:
    T_e.append((u, v, w))
elif v not in U and w < min_weight:
    ...
```

20. Como el algoritmo primero genera el árbol BFS de G , y G está representado mediante matriz de adyacencia, esta operación cuesta $O(V^2)$, luego, iterar por la matrix $V \times V$ toma V^2 operaciones también, luego $O(V^2) + O(V^2) = O(2V^2) = O(V^2)$.

```
bfs_tree = BFS(G) # ->  $M_{V \times V}$  en  $O(v^2)$ 
for i in bfs_tree:
    for j in i:
        if bfs_tree[i][j].w != 0:
            bfs_tree[i][j]
```